

Security architecture (18%)

3.1 Architecture

Architecture and infrastructure concepts

- Cloud
 - Responsibility matrix: Who secures what — provider owns infrastructure , you own data/access
 - Hybrid considerations: On-prem + cloud mix → consistent policies, identity federation
 - Third-party vendors: Evaluate their security, SLAs, right-to-audit
- Infrastructure as code (IaC): infrastructure via config files → version-controlled, automated, scalable
- Serverless: No server management → code-level security only
- Micro-services: Small independent services → each needs its own auth boundary
- Network infrastructure: Routers, switches, firewalls → segment, harden, monitor
- Physical isolation:
 - Air-gapped: No network connection at all — physically isolated
 - Logical segmentation: VLANs/subnets → contain breaches, control traffic
 - Software-defined networking (SDN): Software-controlled network → centrally managed policies
- On-premises: You own & manage everything → full control, full responsibility
- Centralized vs. decentralized: Centralized = easier to secure uniformly; Decentralized = more resilient, harder to manage
- Containerization: App + dependencies in container (Docker) → secure runtime & images

- Virtualization: Multiple VMs on one host → compromised hypervisor = all VMs at risk
- Internet of things (IoT): Weak defaults, rare patches, limited compute → high risk
- ICS/SCADA: Industrial controls (power, water) → disruption = physical consequences
- Real-time operating system (RTOS): Strict timing, embedded/industrial → predictability over security
- Embedded systems: Fixed firmware, hard to patch → compensating controls needed
- High availability: Redundancy + failover → no single point of failure

Considerations

- Availability: Uptime % — can users access it when needed?
- Resilience: Absorb disruption, keep running, recover fast
- Cost: Balance control cost vs. risk reduction
- Responsiveness: Security can't kill performance
- Scalability: Grow/shrink without losing security posture
- Ease of deployment: Automated = less human error, less drift
- Risk transference: Insurance or contract shifts financial impact
- Ease of recovery: Backups + runbooks + tested plans
- Patch availability: Does a patch even exist?
- Inability to patch: Legacy/EOL → isolate it, add compensating controls
- Power: Redundant feeds, UPS, generators
- Compute: Security tools (DLP, encryption) eat resources → plan for it

3.2 Infrastructure

Infrastructure Considerations

- Device placement: Put firewalls/IDS at trust boundaries where they matter most

- Security zones: DMZ / internal LAN / guest → different trust levels, enforce between them
- Attack surface: All entry points → minimize open ports, APIs, interfaces
- Connectivity: Every connection = potential attack vector → limit to necessity
- Failure modes:
 - Fail-open: Device fails → traffic flows through → availability over security
 - Fail-closed: Device fails → traffic blocked → security over availability
- Device attribute:
 - Active vs. passive: Active = blocks/modifies traffic; Passive = watches only
 - Inline vs. tap/monitor: Inline = in the path, can block; Tap = copy of traffic, no impact
- Network appliances:
 - Jump server: Hardened gateway into secure/isolated segments (bastion host)
 - Proxy server: Middleman → filter, cache, hide internal IPs
 - IPS: Monitors + actively blocks malicious traffic
 - IDS: Monitors + alerts only — does not block
 - Load balancer: Distributes traffic → reliability + can offload SSL
 - Sensors: Collect data → feed SIEM/SOAR
- Port security:
 - 802.1X: Must authenticate before getting LAN access
 - EAP: Flexible auth framework inside 802.1X (certs, tokens, passwords)
- Firewall types:
 - WAF: HTTP/HTTPS only → blocks SQLi, XSS, CSRF
 - UTM: All-in-one box → FW + IDS/IPS + AV + VPN + content filter
 - NGFW: Deep packet inspection + app awareness + user identity + threat intel

- Layer 4/Layer 7: L4 = port/IP; L7 = application content (more granular)

Secure communication/access

- VPN: Encrypted tunnel over public network → acts like private network
- Remote access: VPN, VDI, RDP → securing outside-office connections
- Tunneling:
 - TLS: Encrypts data in transit → used in HTTPS, successor to SSL
 - IPsec: Encrypts IP packets → VPNs; transport mode or tunnel mode
- SD-WAN: Software manages WAN links → dynamic routing + integrated security
- SASE: Cloud-delivered → SD-WAN + CASB + ZTNA + FWaaS in one identity-driven framework

Selection of effective controls: Right control for the right risk → proportional, compatible, cost-effective

3.3 Protect Data

Data types

- Regulated: HIPAA, PCI DSS, GDPR → legal requirement to protect
- Trade secret: Formulas, strategies → competitive advantage, must stay confidential
- Intellectual property: Patents, copyrights, trademarks → legal + technical protection
- Legal information: Attorney-client privilege, contracts → strict access + retention
- Financial information: Revenue, payments, audits → regulatory + fraud risk
- Human and non-human-readable: Human = docs/spreadsheets; Non-human = binary, encrypted, machine data

Data classifications

- Sensitive: PII/SPII → always encrypted, strict access
- Confidential: Internal only → disclosure = business damage
- Public: Anyone can see it → protect integrity, not confidentiality

- Restricted: Need-to-know only → highest internal sensitivity
- Private: Personal info (employees, customers) → privacy laws apply
- Critical: Loss = severe operational/safety/national security impact → strongest controls

General data considerations

- Data States:
 - Data at rest: Stored → encrypt it, control access, physical security
 - Data in transit: Moving across network → TLS/IPSec
 - Data in use: In memory/CPU → secure enclaves, app-level controls
- Data Sovereignty: Data lives under laws of where it's stored → affects cloud region choices
- Geo-location: Where data is collected/stored → drives compliance + access restrictions

Methods to secure data

- Geographic restrictions: Enforce where data can be stored/accessed (e.g., EU only)
- Encryption: Scrambles data → only right key can read it
- Hashing: One-way → verify integrity, store passwords (not reversible)
- Masking: Fake but realistic data → safe for dev/test environments
- Tokenization: Replace real data with token → original only in secure vault (used in payments)
- Obfuscation: Make data/code hard to understand → reduces but doesn't eliminate risk
- Segmentation: Isolate data zones → breach in one doesn't expose all
- Permission restrictions: Least privilege → only what's needed, nothing more

3.4 Resilience and Recovery

High availability:

- Load balancing vs. clustering: Load balancing = spread traffic across servers; Clustering = servers act as one unit, failover if node dies

Site Considerations

- Hot Site: Fully live, minutes to failover → most expensive
- Cold Site: Empty space only, hours/days to spin up → cheapest
- Warm Site: Partial setup, periodic backups → middle ground
- Geographic dispersion: Sites in different regions → one disaster can't take out everything

Platform diversity: Different vendors/OS across redundant systems → one vuln can't hit all

Multi-cloud systems: Two+ cloud providers → avoid lock-in, improve resilience

Continuity planning:

- People: Cross-training, succession plans, remote-work capability
- Technology: Backups, tested failover, recovery procedures for critical systems
- Infrastructure: Redundancy, degraded-mode operation, relocation capability

Testing

- Tabletop exercises: Talk through the scenario → find gaps without touching real systems
- Fail over: Actually switch to backup → verify RTO is met
- Simulation: Realistic drill (e.g., fake ransomware) → test response without live impact
- Parallel processing: Run primary + backup simultaneously → validate before cutting over

Backups

- Onsite/offsite: Onsite = fast recovery; Offsite = survives site disaster
- Frequency: How often = your RPO (max acceptable data loss)
- Encryption: Protect backup media if lost/stolen
- Snapshots: Point-in-time copy → fast VM/DB recovery

- Recovery: Must be tested → untested backups aren't real backups
- Replication: Near-real-time copy → minimal data loss, fast failover
- Journaling: Log every change → granular point-in-time recovery

Power

- Generators: Extended outage coverage → needs fuel, takes seconds/minutes to start
- UPS: Instant bridge power → buys time until generator starts or safe shutdown